Investigations of a Constraint Logic Programming Approach to University Timetabling

Czarina Cheng
Le Kang
Norrus Leung
George M. White

Computer Science Dept.
University of Ottawa,
Ottawa, K1N 6N5,
Canada

white@csi.uottawa.ca

ABSTRACT

The casting of university timetables is a problem which combines classical numerical scheduling techniques with important human considerations. It will be argued here that since the application involves the preferences of humans, the problem is qualitatively different than similar problems involving inanimate objects. The humane and profane facets are combined in this study by using the constraint logic programming approach. The constraints are hierarchical: the primary constraints are rigidly enforced and the secondary constraints are relaxed according to their priority if a solution cannot be found. We present a solution based on a Prolog description of the constraints and goals. Two working implementations are described, one using an IBM mainframe and one using a personal computer. Tests with synthetic data and real data from a university have shown that good timetables can be cast using this method in a reasonable amount of time.

1. Introduction

The casting of timetables is an activity which has been the source of a great deal of theoretical and practical work. Since the partitioning of resources among a set of competing processes is a universal problem arising in many different arenas, solutions to this problem are of interest in other domains.

One specialized area is the casting of *school* or *university* timetables *i.e.* finding suitable times and places for professors and students to meet for lectures, laboratories and other academic activities. This area is interesting because the problems are challenging, the results are immediately applicable and data is close at hand.[5, 11, 17, 23, 25, 26, 28, 29, 35]

The limited number of human and material resources available and the restrictions of use placed upon them means that the timetables have to be constrained so that certain conditions are met. Some of the constraints are absolute while others are relative, *i.e.* some constraints *must* always hold while others may be relaxed, if necessary, in order to cast the schedule.

The fact that humans, both students and teachers, are integral parts of the problem leads to the conclusion that the university timetabling problem is qualitatively different from other superficially similar types of timetabling and related problems such as stock-cutting[12] and decryption.[19] It also precludes the use of global optimization methods which may well lead to the "best" use of the resources available as calculated over some synthetic metric but at the cost of severely inconveniencing the personnel involved. It is the human consideration that has led to the approach that was adopted here. It is easier to incorporate human preferences into the formulation of the classical timetabling problem via the constrained logic approach than it is by most other methods. Thus considerations such as "classes should be scheduled only when their teachers are available" and "classes should be scheduled close to their home departments", constraints (7) and (13) of section 4, are easily incorporated into the problem, using the same formulation as that for the common numerical constraints such as no student can take two courses at the same time.

In addition, the constraint satisfaction approach permits the problem specification to be easily changed when constraints are added, removed or modified and allows easy portability among different institutions having different constraint sets.

The vocabulary of educational timetabling is fairly standard, at least within the English speaking world. Unfortunately the words used have ambiguous meanings, causing great confusion when scheduling matters are discussed. To clarify matters, a number of these words are defined below.

Rooms are places where teaching is done. Rooms have certain seating capacities which can be defined in a number of ways. They range from large amphitheatres to small seminar rooms. The rooms may have special furniture in them such as lab benches or pianos, obviously essential for chemistry labs or music lessons. In other cases furniture type is not important and all that is required is a regular lecture hall where everyone can sit down.

Time slots are the quanta of time into which the 24 hour day is divided. There is a great variety of ways in which this can be done. Often the day is divided into time slots of 1 hour each. The day starts in the morning at 8 or 9 am, ends at night at 10 pm and consists of 14 or 13 time slots of 1 hour in length. Often the time slot has a duration of 1.5 hours, 2 hours or 3 hours. The *repetition period* for timetables in usually a week, with successive weeks following each other for perhaps 3 months. Sometimes the repetition period lasts three days or two weeks. The timetable will not change during this repetition period.

A *course* is a body of subject matter which is taught by one or more teachers to one or more students in a room during one or more time slots. Courses usually have names and course codes such as "Introduction to Computing I, CSI1100". Often a single course is split into two or more parallel sections, distinguished from each other by a section letter. Thus CSI1100A and CSI1100B are two sections of the same course. The various sections are concerned with the same subject matter but the material is taught to different groups of students. Two or more sections may or may not have the same teacher but they do not have any

students in common. Occasionally a course has more than one course code which enables administrators to refer to it in more than one way. All the sections of a course usually write a common examination. A course is taught at a rate of so many hours per week, three hours per week being very common. Such a course may occupy 3 time slots of 1 hour's length, 2 time slots of 1.5 hour's length or 1 time slot of 3 hour's length. Other divisions are possible as well and the length of the time slots used can be mixed according to the wishes of the institution. The exact format of the time slots used and the reasons for its choice are a matter of great concern in academic circles.

The *format* of a course's time slots is the pattern of the time slots over the days and hours of the timetable. A course requiring 3 hours per week may be taught in three time slots of one hour's length, each beginning at 9 am on Mondays, Wednesdays and Fridays. Another possibility is 8:30 on Monday, 10:30 on Tuesday and 9:30 on Thursday. Each of these patterns is a format. Often formats are required to lie completely in the morning or completely in the evening. Some commonly used formats can be found in[5, 11, 25]

A *program* of studies is a set of courses which students must follow. This program is broken down chronologically into *years* and *terms*. Students enrolled in Computer Science begin their studies by taking a certain set of courses offered in the Fall term. This is followed by another set of courses in the Winter term and so on for four complete years. The program for these first year students in the Fall session is a logically complete unit. However, this does not imply that this program is completely independent of other programs. Students have certain common required courses shared with students in other programs and certain other courses which they alone must take. These course requirements that spread across a number of different programs cause the greatest problems in the casting of the timetables.

Different institutions vary tremendously in programs, courses and formats. They differ also in the requirements that they impose on these items. This is the major cause of difficulty in using machine generated solutions in different institutions.

2. Problem Definition

Many techniques have been brought to bear on the timetabling problem but there are no generally accepted solutions in existence yet. Far more success has been obtained for various subsets, notably the *examination scheduling* problem.[2, 8, 18, 32, 33] and the *classroom assignment* problem.[9] The difficulty of casting timetables is complicated by the inherent complexity of the problem which is known to be NP-hard.[10] Solutions can involve thousands of values of 3 or more variables.

The university timetabling problem can be expressed as follows:

Let there be m time periods, $k = 1..m$; n_2 teachers, $j = 1..n_2$; n_1 classes, $i = 1..n_1$; and n_r rooms, $l = 1..n_r$. There is a requirements matrix $R = (r_{ij})$ and two vectors. E is an n_1-vector whose *ith* element is the enrollment in class i and C is an n_r-vector whose *ith* element gives the capacity of room i. Find a function

$$S = \{ x_{ijkl} \}: \{1..m\} \times \{1..n_1\} \times \{1..n_2\} \times \{1..n_r\} \rightarrow \{0,1\}$$

where $x_{ijkl} = 1$ iff teacher j meets class i in room l during period k such that:

$$\sum_l \sum_k x_{ijkl} = r_{ij} \tag{1}$$

$$\sum_j x_{ijkl} \leq 1 \tag{2}$$

$$\sum_i x_{ijkl} \leq 1 \tag{3}$$

$$\sum_l x_{ijkl} \leq 1 \tag{4}$$

$$\sum_j \sum_k e_i \; x_{ijkl} \leq \sum_j \sum_k C_l \; x_{ijkl} \tag{5}$$

(1) requires that the number of meetings of class i and teacher j conforms to the specifications of the requirements matrix;
(2) specifies that each period-class-room has at most one teacher involved;
(3) specifies that each teacher-period-room has at most one class in it;
(4) specifies that each teacher-period-class occupies at most one room;
(5) specifies that all rooms are large enough to contain the classes scheduled into them.

Any particular institution will likely have other requirements in addition to these basic ones. This is discussed in section 4.

3. Approaches

This problem has been studied for a long time, primarily by university based researchers and education administrators because the problem arises constantly and data is readily available. The solutions obtained to date have not been entirely satisfactory due to the complexity of the problem. To obtain a usable solution, researchers have generally had to simplify the problem, often to the point where the solved problem bears little resemblance to the real-life situation that originally spawned it. Several different approaches have been used to formulate solutions. A bibliography of some past explorations can be found in the review by Schmidt and Ströhlein.[26] Some of the approaches to solving the timetabling problem are discussed below.

The celebrated *graph colouring problem* requires that the nodes of a graph be coloured in such a way that no two adjacent nodes, *i.e.* those connected by an edge, have the same colour. The problem requires that the minimum number of colours be used. By identifying each of the nodes with a course and each pair of courses taken by a student as an edge, the minimum number of colours corresponds to the minimum number of periods required to timetable the courses. This problem has been studied both in its own right and because of its application to the timetabling problem.[4,6,13,24,33]

The timetabling problem considered as a problem of reducing the number of constraint violations becomes an *optimization problem*. Unfortunately a large number

of variables and constraints are required in the description of realistic problems and this can cause the solution space to become unmanageably large. However several models have been studied and some good results have been obtained.[9,15,22,27]

In the *interactive approach*, a semi-automated algorithm accepts data incrementally from an operator and builds a timetable piece by piece. Systems can be equivalent to electronic spreadsheets[29] or have some intelligence built in.[34] There are several methods in common use which do not use computers at all but rely entirely on humans with great stamina and long pencils.[5,20,28]

A new approach which has only recently been applied to the timetabling problem is that of *simulated annealing*, based on a Monte Carlo scheme. Courses are placed in the timetable at random and an initial cost is calculated for this initial placement. Random subsets of courses are then randomly re-arranged and for each new placement, a new value of the cost function is calculated. If the new cost is lower, the new placement is kept. If the new cost is higher, the new placement may still be kept based on chance and the value of the so-called temperature. The process continues with progressively lower values of temperature until either the cost is reduced to zero or a certain limiting number of iterations is reached.[1]

Another recent innovation is the use of the so-called *TABU* search family of algorithms.[16] An initial solution which may or may not satisfy all the constraints is formulated as the starting case and the initial value of an objective function is calculated. Values of the variables are changed and new values of the objective function are calculated. These iterations are halted when some stopping criterion is met. During the iterations, trial solutions which violate some of the constraints are encountered but provisionally accepted. Values of the variables are then changed in the hope that the new values will bring the solution back into the feasible region within a small number of iterations.

The *agent based* class of algorithms models human agents and attempts to mimic the procedures used by human beings, possibly using some of the strategies outlined above.[36] Very little is known presently about the quality of results generated by these procedures.

4. Constraint Logic Programming

Constraint programming is a relative newcomer to the area of timetabling although some work has been done in related areas by Carmel and Itzovitz[7] whose Comprehensive University Planner (CUP) is designed to support all administrative activities related to a university teaching system. As well, Feldman and Golumbic[14] have reported on a project of using constraint satisfaction algorithms for interactive student scheduling. The project was to determine which algorithms perform best in solving the student scheduling problem and under what conditions. They concluded that the order in which the variables were assigned values significantly affects the performance of the algorithms. After testing three algorithms, regular backtracking, forward checking and word-wise forward checking, they concluded that with large numbers of courses the word-wise algorithm was clearly better than the others, with regular backtracking the worst.

Yoshikawa *et al.*[37] have investigated the related problem of timetabling high schools using a general purpose constraint relaxation problem solver called COAS-TOOL. The method used was to create an internal relaxation assignment using the Really-Full-Lookahead Greedy (RFLG) algorithm, followed by a strongly biased optimization algorithm, the Min-Conflicts Hill-Climbing (MCHC) algorithm described by Minton *et al.*[21] These tools and techniques successfully produced timetables in three cases that were studied. The execution time reported varied between 26.2 minutes to 2.8 hours and the authors observed that the solution occasionally fell into an unacceptable local optimum. Otherwise the results were acceptable.

These results, while interesting, are not directly comparable to the results reported here because our problem is larger and the constraint set is different, making the granularity of our problem considerably different from theirs. This illustrates one of the differences between secondary school and university scheduling. In the schools, a group of students, typically called a *class* stay together for the entire week. These classes can then be timetabled independently apart from room considerations.

The rule base forming the non-volatile constraints used by the program described in this paper is divided into those rules which *must* be satisfied and those which *may* be satisfied. Any timetables produced will always satisfy the primary constraints and as many secondary constraints as possible.

4.1. Primary Constraints

1) No two or more required courses of a student program can be scheduled at the same time.
2) No recommended optional course of a student program can be scheduled at the same time as any required course of the same student program.
3) No two or more courses taught by professor can be scheduled at the same time.
4) Day courses must be scheduled during the day and evening courses must be scheduled during the evening.
5) The classrooms assigned to courses must have capacities equal to or larger than the enrollments of the courses scheduled into them.
6) Classrooms must not be scheduled during those times they have been reserved for other uses.
7) Classes cannot be scheduled when their professors are unavailable.
8) Preassigned courses must be scheduled into their preassigned classrooms at their preassigned times. There are three different kinds of preassignment. The first case occurs when a course's teaching time and classroom are all fixed before the scheduling; these are completely preassigned courses. The second case occurs when only the teaching time of the course is fixed; these are called time-preassigned. The final case preassigns a certain classroom to a course.
9) Certain courses must be taught in the same classroom at the same time. This is required when a course has more than one course code, as happens sometimes when a course is taken by students from different departments or faculties.

10) Certain courses must overlap in their teaching times.

11) Certain courses should have exactly the same teaching time but different classrooms. This constraint and the one above can be used to enforce course prerequisites.

12) If a course is held more than once a week, there will always be at least one day between any two lectures of the course.

4.2. Secondary Constraints

The following list of constraints will be imposed if it is possible to do so. Some of these have been added for improving the resource utilization and some are present for improving students' course choice. If it proves impossible to create a schedule with all secondary constraints, they are relaxed in groups in order to decrease the program's run time.

13) Courses should be scheduled into classrooms in the buildings as close as possible to a designated building. The first attempt will be the preferred building. If this is infeasible, then the next and succeeding buildings will be tried until a success is found.

14) Courses should be scheduled into rooms of the specified type. The *type* property of a room has three parameters: class type, furniture type and seating type. Any or all of the types can be specified. The system will first try to find rooms satisfying all three arguments. If they cannot be found, rooms which satisfy room type and furniture will be sought. If this fails, rooms satisfying only room type will be sought. If this fails, the constraint is abandoned.

15) Courses should be scheduled into classrooms whose size is large enough (but not too large) to contain them. The classroom should be in the smallest size range possible. Note that this is not the same as constraint 5.

16) Recommended optional courses of a student program should be scheduled in such a way that there are the fewest time conflicts among them.

17) Courses should be scheduled into the time zone specified by the department or the professor. The time slots available for day courses are partitioned into three time zones, morning, noon and afternoon. While evening courses are always scheduled into the evening time zone, a day course should be scheduled into the prescribed day zone if possible.

18) Friday afternoon shall be used only after all other scheduling possibilities are exhausted.

19) A professor should have at least one time slot free between any two teaching time slots.

Some of the constraints are at least partially in conflict with one another. This is particularly true with constraints 13 and 15 which deal with geography and room size. Constraint 15 will attempt to put a class of students into a classroom just large enough to hold them. This leads to efficient use of classrooms but may require that students and professors do a lot of walking between classes to get to those class-

rooms. Constraint 13 will minimize the amount of transportation (at least for professors) but may lead to an inefficient use of classrooms.

An analysis of the University of Ottawa's classroom size distribution yields the values shown in figure 1. The *Range* is a code assigned to a particular range of room and course sizes. As in many institutions, the room size distribution does not correspond well with the observed distribution of course sizes.

Room Size Distribution		
Room Size	Range	Number of Rooms
1-10	1	9
11-18	3	17
19-30	5	44
31-50	7	31
51-70	9	35
71-100	11	18
101-150	13	4
151-200	15	5
201-300	17	2
>300	19	2

Fig. 1. Classroom Size Distribution

As a result, in order to schedule all the courses offered, smaller courses must be scheduled into larger classrooms. The permitted difference between room size and class size is specified by a *size limitator.* Each size range is associated with a size limitator whose value indicates the largest classrooms into which courses whose enrollments fall into the size range may be assigned. A course with enrollment R whose size limitator is L can only be scheduled into classrooms whose size is no larger than range $R + 2 \times L$. A typical size limitator table is shown in figure 2.

Size Limitators										
Range	1	3	5	7	9	11	13	15	17	19
Size Limitator	1	1	2	1	2	1	1	1	1	0

Fig. 2. Size Limitator Table

This size limitator instance gives a value of 1 to rooms in range 1, a value of 2 for rooms in range 5 and a value of 0 for rooms in range 19. The larger the value of the size limitator, the greater the "flexibility" in the placing of a class whose size is in the specified range. Those ranges where the mismatch between class size and room size is the greatest have the largest values of size limitator. The size limitator has to be tuned for each institution and year in order to yield an efficient utilization of rooms.

The 19 constraints listed above are written as the conjunction of sub-rules and typically have the form shown in figure 3.

These rules are interpreted by a modified Prolog backtracking algorithm based on depth first search. The classical method is not efficient enough for our purposes and modifications were necessary to find good solutions within a reasonable execution

time. After much experiment, it was concluded that the order in which constraints should be considered is the following:

- select a time block in the specified time zone
- select a classroom with the proper room type
- verify whether this time-classroom pair violates any constraints

No required courses of a program may be scheduled at the same time

for all i,j,k

no_common_member(T_j, T_k) $\leftarrow$

is_a_member($sp_i(s_1, s_2)$, S) **and**

is_a_member(c_j, s_1) **and**

is_a_member(c_k, s_1) **and**

$c_j \neq c_k$ **and**

teaching_time(c_j, T_j) **and**

teaching_time(c_k, T_k)

where $i = 1, \ldots, m$ $\quad j = 1, \ldots, n$ $\quad k = 1, \ldots, n$

where

T_i is the time slot of $course_i$

sp_i is the set of courses of the ith program

s_1 is a list of required courses

s_2 is a list of optional courses

Fig. 3. One of the Constraints

If no constraints are violated, the course is scheduled into that classroom during that time block. If not, backtracking is performed with different classrooms and different time blocks. It may happen that no solution can be found even after attempts using all suitable time blocks and classrooms. The secondary constraints have been arranged in a constraint hierarchy[3] and if a course cannot be timetabled, these constraints are relaxed one by one and a new attempt is made. The highest numbered constraint, number 19, is the first constraint to be relaxed *i.e.* has the lowest priority. The lowest numbered of the secondary constraints has the highest priority and is the last to be relaxed.

There is a possibility that even after all secondary constraints are relaxed, a course is still not scheduled. If this occurs, the program enters a procedure that we call *equivalent reversing*, or ER. When a course becomes unschedulable, the ER first chooses an equivalent course which has already been successfully scheduled. It then deletes the latter's assignments and tries to reschedule it after first scheduling the former. If the former cannot be scheduled after deleting the latter's assignments, the ER will replace the latter's assignments and choose another equivalent course and repeat the procedure. Equivalent courses are those which:

- have enrollments in the same size range
- are in the same time zone
- require the same room type and
- request the same building

To avoid long search paths, nested equivalent reversing is forbidden *i.e.* there is no invoking an equivalent reversal from within an equivalent reversal. In addition, the number of equivalent reversals that can be selected for any course has been limited to 4. Of course, assignments to completely prescheduled courses cannot be deleted. If the program cannot schedule a certain course even after the equivalent reversing procedure, it is placed in a list of unschedulable courses to be manually placed later. The goal is to minimize the number of elements in this list.

5. Information Management

The information that is considered at different stages of the problem comes from different sources and has different degrees of volatility.

Some of the knowledge, such as the room inventory which contains lists of all the rooms available along with their sizes and facilities such as furniture type, overhead projectors and handicap access etc., is relatively stable while other blocks of knowledge, such as professor-course pairs, student program lists and professor unavailability lists, are quite volatile and change greatly from year to year, often changing several times within the same year. All this knowledge must be used by the system and if any of it changes, the changes must be accommodated without destroying the parts of the solution which are satisfactory. This is done by the three stage system described in detail later. All the data are gathered during the first stage when the system knowledge is created in the most up-to-date form possible. The constraint program is used during the second stage, taking the knowledge prepared by the first stage. Changes in the knowledge base are treated in the third stage. The timetable prepared by the constraint program is modified incrementally by hand as changes in the knowledge base arrive, keeping the constraints satisfied. Thus when a new academic session starts, the new timetable can be used immediately because it reflects the latest changes known to it.

Another aspect is the distributed nature of the knowledge.[36] This distribution not only reflects the different resources of the various parts of the university but also incorporates their different world views of the situation. Some faculties take the position that all their staff are available for duty during the whole week while other faculties wish their staff to be able to concentrate their teaching into two or three days of the week, leaving the other days free for other activities. There are, therefore, relatively few "universal rules" which can be applied across the entire problem. This is further complicated by the fact that students must take courses from a spectrum of disciplines and are therefore subject to different rules at different times. This problem is tackled by partitioning the domains of knowledge into a number of disjoint subdomains, each one of which is sufficiently homogeneous to be described into a small number of rules. In some cases this is best handled by incorporating a list of exceptions into the knowledge base. Typically, this exception list consists of professors and the times they do not wish to teach and a list of rooms and the times they are unavailable.

At the University of Ottawa, information is captured by the chief timetabling officer and by departmental administrative personnel. In this first stage, volatile data

concerning academic staff and student programs is entered in a human readable format using any available word processor, the only requirement being that ASCII output, with no embedded formatting commands, can be produced. This data must be independently verified to make sure it is correct. One recurring problem is the overspecification of courses. In the rule base, there is a rule which tries to make optional courses available to as many students as want to take them. These courses have a tendency to appear in the optional course lists of many different departments, making them difficult to schedule.

In the second stage, this information is transformed into Prolog predicates. These predicates are entered into the main inference engine to produce the timetable.

The third part of the system consists of a perturbation module which allows the chief timetabling officer to modify the timetable based on knowledge updates. This part is required because the timetable must be generated during the month of March for the academic sessions which begin in the following September and January. Much can happen in the intervening period but the constraint resolving part cannot be re-executed. Thus the timetable is continually updated as new information is produced, in such a way as to keep most of the timetable intact. The perturbations are introduced manually and verification with the existing constraints is done by hand.

In cases where important changes take place, *e.g.* a professor leaves the university or a building burns down, the program can be rerun keeping everything the same except for the new variable value causing the perturbation. This permits changing the classrooms while keeping the times for all the courses the same as they were. An academic unit which already has a satisfactory timetable can keep it from year to year while other units can change to adapt to their changing resources.

6. Implementations

Two implementations of this system currently exist. The older one runs on an IBM mainframe under VM/CMS. The knowledge base and inference engine are written in a variant of the prolog language, WPROLOG.[30]

When the data has been prepared and checked, the main WPROLOG engine accepts it and generates the timetable. The output of this process is then input to other modules which generate human readable timetables that can be distributed to students, professors, resource administrators and printed in the official university timetable book. The information is classified in a number of ways:

- by course
- by department
- by room
- by professor.

The output can also be downloaded to other programs for further processing, if desired.

A newer implementation uses a stand-alone PC platform. Development of this version has been done using a 33 MHz - 80486 chip, using MS-DOS version 5.0 and the ALS Prolog[31] system. The data is captured in the same format. The timetabling

officers can use any editor they wish, provided that it can produce ASCII output. The data transformations, the main prolog engine and the final data formatting and printing are done on the same PC. The user controls the system via a windows-like user interface using pop-up menus and mouse clicks.

The input and output are identical for the two systems.

7. Sample Results

One of the system goals is to produce timetables that use resources efficiently. A measure of room utilization efficiency is the *average seating usage*, ASU. The ASU is equal to the mean percentage of classroom space occupied by students during those time slots when the classroom is used.

A set of test data used in the production of the time table by traditional means was used as input data. There were 2147 course entries with 155 preassignments included. The room inventory included 167 classrooms whose sizes ranged from 4 to 474. The size distribution of the enrollments of the courses is shown in figure 4 in terms of the range values defined in figure 1.

Distribution of Enrollment										
Range	1	3	5	7	9	11	13	15	17	19
Number of Entries	156	160	871	387	267	189	63	30	21	3

Fig. 4. Size Distribution of Course Enrollments

Using the size limitators of figure 2, the results shown in the second line of figure 5 were obtained. An additional test run with more data is summarized on the third line of figure 5. These figures were obtained with the mainframe implementation. Note that the value of ES in line 2 differs from the number of course entries because of course numbering duplication, the phenomenon described by constraint 9.

Test Results				
CPU(secs)	PE	ES	UE	ASU
1402	155	2113	0	80.2%
9394	155	2244	24	88.8%

where:
CPU(secs) - CPU time in seconds
PE - number of preassigned courses
ES - number of courses scheduled
UE - number of unscheduled courses
ASU - average seating usage

Fig. 5. Test Run Results

The values of the average seating usage, ASU, for various room size ranges are shown on the histogram of figure 6. The shaded bars are the values corresponding to the traditional manual system and the clear bars show the ASU for the computerized

system. It is interesting to see that although the computerized system was not designed to minimize the ASU, it did produce substantially better values than did the manual system. Furthermore, this was done while simultaneously enforcing the constraints imposed on the timetable, not all of which were satisfied when the manual system was used.

With the size limitator of figure 2, all the courses were scheduled. This, however, says nothing about the location of the rooms. In order to minimize the amount of transportation, the size limitators have to have values larger than the minimum required for a successful timetable yet be small enough to improve the existing measures of efficiency. The balance between space efficiency and minimum transportation is one of the factors which must be weighed by the system operators.

A second measure of room utilization efficiency is the *average room utilization*, ARU, of a room calculated by dividing the total number of hours in the week avail-

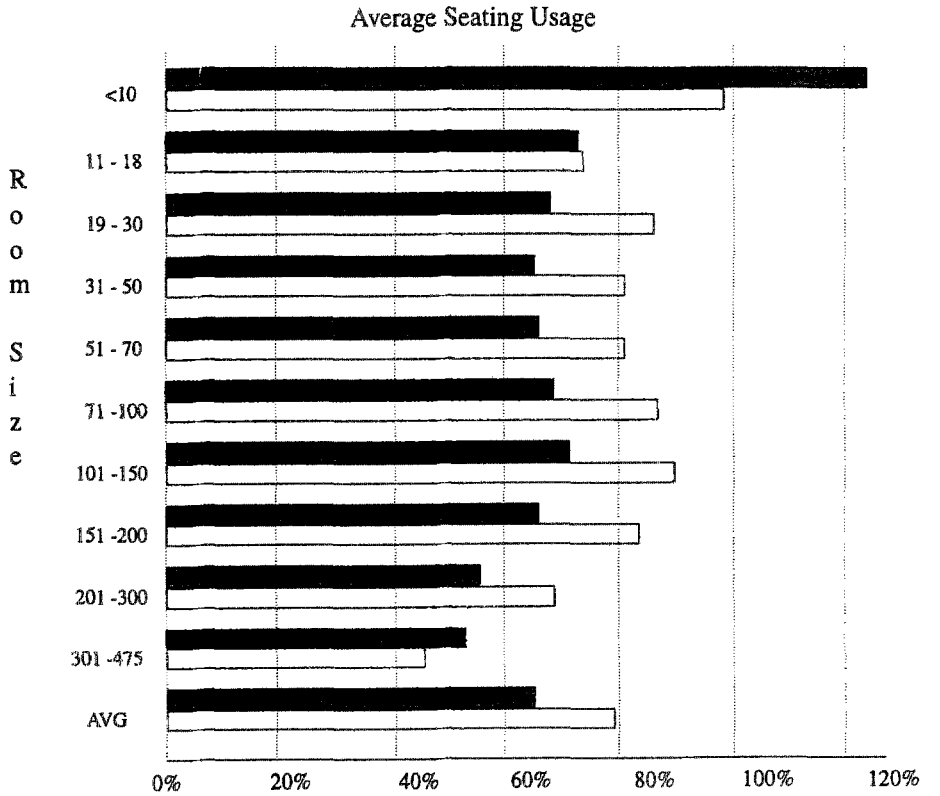

Fig. 6. Values of Average Seating Usage, ASU

Shaded bars are results with manual system.
Clear bars are results with computerized system.

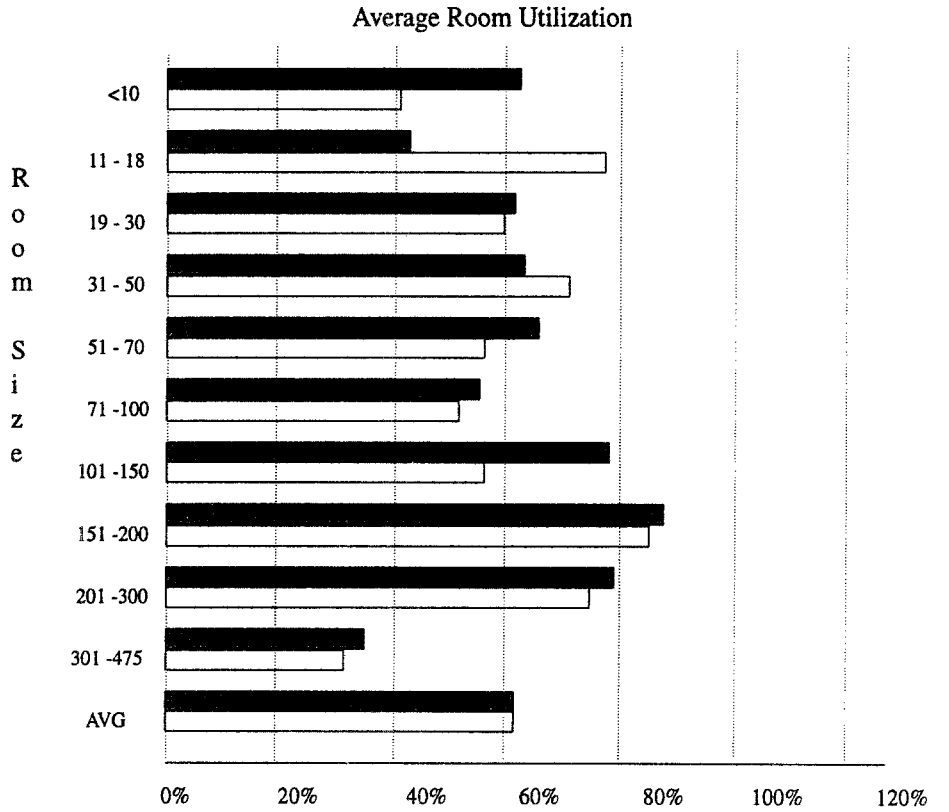

Fig. 7. Values of Average Room Utilization

Shaded bars are results with manual system.
Clear bars are results with computerized system.

able for use by the total number of hours in the week during which it was actually used. A histogram of these values for various room size ranges is shown in figure 7.

It can be observed that the manual system tends to use the smallest room size range more often than does the computerized version and use the next largest room size range less often than does the computerized version. Apart from this, the ARU values are about the same. This distribution of ARU values is of interest to planners who must make efficient use of the physical plant inventory. Figure 7 suggests that rooms in the capacity range 301 - 475 are used less often than others and that some of them might be converted into laboratories or offices if required.

One other timetable check that was made concerns the starting time of the courses. The manual method tends to schedule courses more heavily towards the early

afternoon rather than in the early morning or late afternoon, reflecting the current professor preferences. One problem of computerized scheduling systems is that course starting times are weighted by the order in which the possible values are tried. This usually means that early mornings time slots will be more heavily used than the others since these time slots are tried first.

The inference engine used in this system chooses trial time slots according to a probability density function which approximates the one actually observed by the present manual system. Thus the distribution of course starting times is observed to be about the same in both the manual and the computerized versions.

8. Discussion

The system has been thoroughly tested with both artificial data and real data obtained from the university Registrar's office. Quantities of data representing departments, faculties and the entire university have been used to generate real timetables. In doing so, several additional problems had to be solved, in addition to the ones described earlier.

The manual timetabling method tolerated a number of different course formats. The traditional three hours per week was delivered as one 3 hour block, two 1½ hour blocks or three 1 hour blocks. Because other block formats were used by certain departments, a total of 15 different formats were identified in use at one place or another. In order to rationalize the use of classrooms a standard 1½ hour time slot is now used which can be doubled to make a single 3 hour time slot. A two hour time slot can be used if the professor wishes to begin at 8:00 in the morning. This will not interfere with the orderly allocation of 1½ hour time slots which begin at 8:30 am and continue throughout the day.

Another subtle but important aspect which must be considered is the political implication of the system. Unlike job shop scheduling, this program orders many aspects of the life of human beings. Fears of the "big brother" surface from time to time bedeviling relations between management and union, staff and timetabling officers, and humans and machines.

9. Conclusion

Our experience with these programs, spanning many years, leads us to conclude that:

1) All courses, both required and optional, of all programs in the university can be scheduled by machine with the resources available.
2) With the present room inventory, the size limitator can be adjusted such that all courses are scheduled without excessive movement between classes.
3) The number of recommended optional courses for students from different departments can been systematically increased. This was not generally attempted with the present manual system.

4) The former rate of seating usage, about 65%, has been increased to about 80%. Thus the efficiency of resource usage can be increased even while reducing movement between classes.
5) The large amounts of work required by the manual timetabling process can be greatly reduced.
6) The amount of CPU time used, less than 25 minutes for a typical run, is quite practical. It should be noted, however, that the time required may be increased by an order of magnitude as resources become scarce.

Work is continuing on improving rules corresponding to certain types of rather specialized requests made by the teaching staff and on improving the user interface.

10. Acknowledgements

We were greatly assisted in this work by the Registrar of the University of Ottawa, Mr. George H. von Schoenberg and by the timetabling officer, Ms. Pauline Bélanger. We wish to thank Prof. Clarence (Skip) Ellis for his hospitality at the University of Colorado during the drafting of this manuscript.

References

1. D. Abramson, "Constructing School Timetables Using Simulated Annealing: Sequential and Parallel Algorithms," *Management Science*, vol. 37, no. 1, pp. 98-113, Jan. 1991.
2. Nagraj Balakrishnan, Abilio Lucena, and Richard T. Wong, "Scheduling Examinations to Reduce Second-Order Conflicts," *Computers Ops. Res.*, vol. 19, no. 5, pp. 353-361, 1992.
3. Alan Borning, Bjorn Freeman-Benson, and Molly Wilson, "Constraint Hierarchies," *Lisp and Symbolic Computation*, vol. 5, pp. 223-270, 1992.
4. D. Brelaz, "New Methods to Color the Vertices of a Graph," *Comm. A.C.M.*, vol. 22, no. 4, pp. 251-256, 1979.
5. J.E. Brookes, *Timetable Planning*, Heinemann Educational Books, London, U.K., 1980.
6. J. Randall Brown, "Chromatic Scheduling and the Chromatic Number Problem," *Management Science*, vol. 19, pp. 456-463, 1972.
7. C. Carmel and M. Itzovitz, "A Comprehensive University Planner Implemented in a 5th Generation Language," *Proceedings of 33rd Annual College and Administrative Comptuer Users Conference*, pp. 418-428, Los Angeles, Calif., 1988.
8. M.W. Carter, "A Survey of Practical Applications of Examination Timetabling Algorithms," *Operations Research*, vol. 34, no. 2, pp. 193-202, Mar-Apr. 1986.
9. Michael W. Carter, "A Lagrangian Relaxation Approach to the Classroom Assignment Problem," *INFOR*, vol. 27, no. 2, pp. 230-245, 1989.

10. M.W. Carter and C.A. Tovey, "When Is the Classroom Assignment Problem Hard?," *Working Paper #89-03*, Department of Industrial Engineering, University of Toronto, 1989.

11. Richard A. Dempsey and Henry P. Traverso, *Scheduling the Secondary School,* Nat. Assoc. of Secondary School Principals, Reston, Virginia, U.S.A., 1983.

12. M. Dincbas, H. Simonis, and P. van Hentenryck, "Solving a Cutting-Stock Program in Constraint Logic Programming," *Proc. of Fifth Int. Conf. and Symp. on Logic Programming*, vol. 1, The MIT Press, Cambridge, Massachusetts, 1988.

13. F.D.J. Dunstan, "Sequential Colorings of Graphs," *Proceedings of the 5th British Combined Conference*, pp. 151-158, Aberdeen, England, 1975.

14. Ronen Feldman and Martin Charles Golumbic, "Interactive Scheduling as a Constraint Satisfiability Problem," *Annals of Mathematics and Artificial Intelligence*, vol. 1, pp. 49-73, 1990.

15. J.A. Ferland and S. Roy, "Timetabling Problem for University as Assignment of Activities to Resources," *Computers & Operations Research* , vol. 12, no. 2, pp. 207-218, 1985.

16. A. Hertz, "Tabu Search for large scale timetabling problems," *Eur. J. Op. Res.*, vol. 54, pp. 39-47, 1991.

17. Le Kang and George M. White, "A Logic Approach to the Resolution of Constraints in Timetabling," *Eur. J. Op. Res.*, vol. 61, no. 3, pp. 306-317, 1992.

18. Vahid Lotfi and Robert Cerveny, "A Final-exam-scheduling Package," *J. Opl. Res. Soc.*, vol. 42, no. 3, pp. 205-216, 1991.

19. Michael Lucks, "A Constraint Satisfaction Algorithm for the Automated Decryption of Simple Substitution Ciphers," *Lecture Notes in Computer Science*, vol. 403, pp. 132-144, Springer-Verlag, 1990.

20. Rick Mayoh, "Mission Impossible: Drawing up NHL Schedule," *Ottawa Citizen*, Ottawa, Canada, Jan. 15, 1992.

21. Steven Minton, Mark D. Johnston, Andrew B. Philips, and Philip Laird, "Minimizing Conflicts: A Heuristic Repair Method for Constraint Satisfaction and Scheduling Problems," *Artificial Intelligence*, vol. 58, no. 1-3, pp. 161-205, Dec. 1992.

22. J.M. Mulvey, "A Classroom/Time Assignment Model," *Eur. J. Op. Res.*, vol. 9, pp. 64-70, 1982.

23. J.E.L. Peck, R. Osterman, and D. De Werra, "Some Experiments with a Timetabling System," *OR Spektrum*, vol. 3, pp. 199-204, 1982.

24. J. Peemoller, "A Correction to Brelaz's Modification of Brown's Colouring Algorithm," *Comm. A.C.M.*, vol. 26, pp. 595-597, 1983.

25. Anthony Saville, *Instructional Programming: Issues and Innovations in School Scheduling,* Charles E. Merrill Publishing Co., Columbus, Ohio, U.S.A., 1973.

26. G. Schmidt and T. Ströhlein, "Timetable construction - an annotated bibliography," *Comp. J.*, vol. 23, no. 4, pp. 307-316, 1980.

27. S.M. Selim, "An Algorithm for Producing Course and Lecture Timetables," *Comput. Educ.*, vol. 7, no. 2, pp. 101-108, 1983.

28. Gardner Swenson, Donald Keys, and J. Lloyd Trump, *Providing for Flexibility in Scheduling and Instruction,* Prentice-Hall, Inc., Englewood Cliffs, N.J., 1966.

29. E.M. Timmreck, "Scheduler - A Program that Uses Instructor and Student Preferences to Form Timetables," *Comp. Sci. Tech. Report #3*, Univ. of Wisconsin, 1967.

30. R.G. Veitsch and J.C. Wilson, *An Introduction to PROLOG Programming with WPROLOG - Univ. of Waterloo PROLOG Interpreter for VM/SP CMS,* WATCOM Products, Inc., Waterloo, Canada, 1986.

31. Christopher M. White and Kenneth A. Bowen, *ALS Prolog User's Guide and Reference Manual*.

32. George M. White and Pak-Wah Chan, "Towards the Construction of Optimal Examination Schedules," *INFOR*, vol. 17, no. 3, pp. 219-229, 1979.

33. George M. White and Michel Haddad, "An Heuristic Method for Optimizing Examination Schedules Which Have Day and Night Courses," *Comput. Educ.*, vol. 7, no. 4, pp. 235-238, 1983.

34. George M. White and Simon K.S. Wong, "Interactive Timetabling in Universities," *Comput. Educ.*, vol. 12, no. 4, pp. 521-529, 1988.

35. W. Deane Wiley and Lloyd K. Bishop, *The Flexibly Scheduled High School,* Parker Publishing Co., West Nyack, N.Y., 1968.

36. Michael Woitass, "Coordination of Intelligent Office Agents - Applied to Meeting Scheduling," *Proc. of the IFIP WG8.4 Conf. on Multi-User Interfaces and Applications*, pp. 371-387, Elsevier Science Publishers, Sep. 1990.

37. Masazumi Yoshikawa, Kazuya Kaneko, Yuriko Nomura, and Masanobu Watanabe, "A Constraint-Based Appeoach to High-School Timetabling Problems: A Case Study," *Proc. Twelfth Nat. Conf. on Artificial Intellegence (AAAI-94)*, Seattle, WA, Jul. 31-Aug. 4, 1994.